EECE430 – Software Engineering

AI Inventory Agent

Inventory Agent

In this lab you will build a simple AI agent that manages a warehouse inventory through a chat interface. The application uses Django and SQLite for the database and web interface, and Google's Gemini API for the model. You will define the inventory operations as Python functions, describe them to the model, and implement the loop that executes the model's requested calls and returns the results. By the end, you will have a working web application where typing a request such as "add 20 screwdrivers" or "how many hammers are in stock?" causes the agent to select and execute the correct database operation.

Font colors used in this :

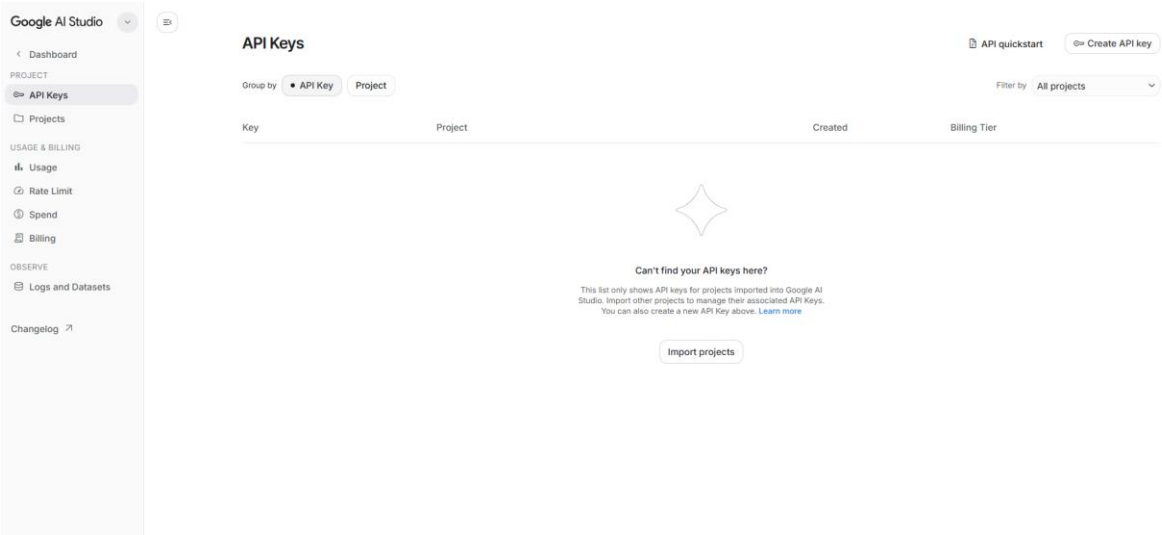
- The light blue font is for commands you have to execute.
- The light orange font is for files to study their content.
- The red color with highlight is used for remarks.

Step 1: Get your Gemini API key

An LLM API key is a password that lets your code talk to your chosen AI model. Usually, you need to pay and add credits for your key but for the sake of this tutorial we will use the Google Gemini model as it offers a free tier. Please note that you should never make your API key accessible to other users; you must never share it, commit it to github or make it easily accessible. If your API key ever gets leaked, delete it immediately and create a new one.

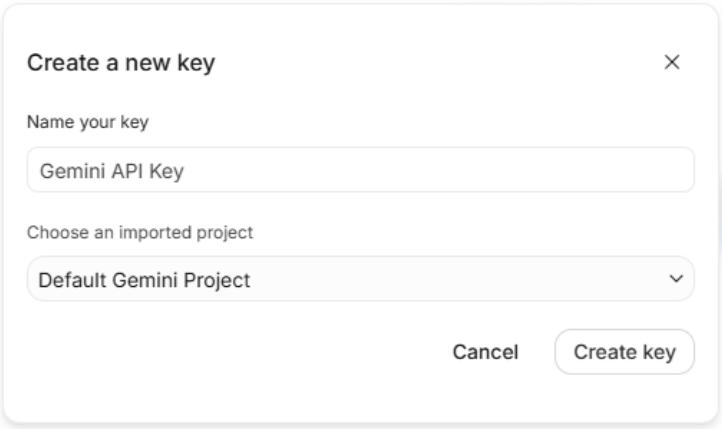
To get your Google Gemini API key:

1. Go to https://aistudio.google.com/apikey
2. Sign in with a Google account. Any personal Gmail account works.
3. If it asks you to accept the Terms of Service, tick the boxes and continue.

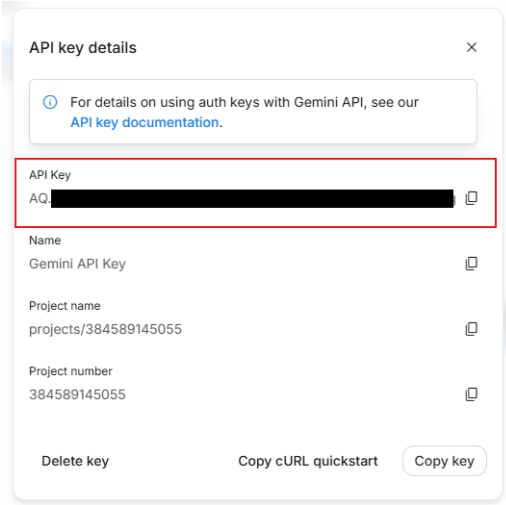


4. Click the Create API key button (Top right corner).

5. Set “Choose an imported project” to “Default Gemini Project”. It is generally recommended to rename your key to keep track of which key belongs to which project. For the sake of this tutorial, we will go with Google’s default parameters.



6. Click “Create Key”. A long string will appear, click the “Copy key” button or the copy icon next to the key.



7. Paste it somewhere safe for now: a text file on your desktop, or a note. You'll need it in a few steps. If you lose this key you won’t be able to see it again. You must create a new one.

Step 2: Project folder, virtual environment and packages

1- Create your project folder, for the sake of this tutorial we will name it “Inventory_agent”.
2- Open CMD and navigate to your folder, then create a virtual environment:

Windows:

```
python -m venv venv
venv\scripts\activate
```

MAC/Linux:

```
source .venv/bin/activate
```

3- Install the 3 packages we need:

In Cmd run: pip install django google-genai python-dotenv

- django — the web framework and the database layer
- google-genai — the official Gemini SDK
- python-dotenv — reads your API key from a file so it never touches your code

Step 3: Create the Django Project

Django gives us a command that generates a project skeleton: a settings file, a URL router, and a management script. Rather than writing these from scratch, we generate them and then read through what was created so nothing feels like magic.

We're using Django for three reasons: it comes with SQLite already configured, so we get a database with zero setup; it gives us a built-in admin panel to inspect our inventory; and it serves our chat page. One framework instead of three tools.

To generate the project skeleton, in CMD run: django-admin startproject config .

The dot (.) will create the project in the current directory.

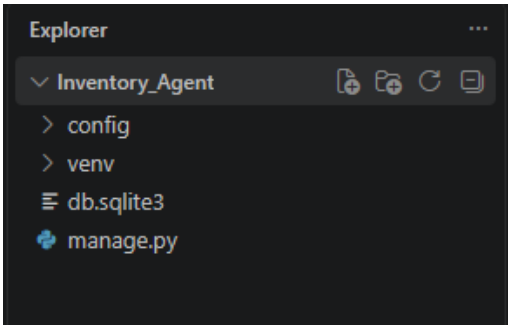
Then run: dir (Windows), ls (MAC)

```
(venv) C:\Users\Ali\Desktop\inventory>dir
 Volume in drive C has no label.
 Volume Serial Number is D652-E66F

 Directory of C:\Users\Ali\Desktop\inventory

09/11/2026  05:08 PM    <DIR>          .
09/11/2026  05:08 PM    <DIR>          ..
09/11/2026  05:08 PM    <DIR>          config
09/11/2026  05:08 PM               684 manage.py
09/11/2026  02:40 PM    <DIR>          venv
               1 File(s)            684 bytes
               4 Dir(s)  341,705,269,248 bytes free

(venv) C:\Users\Ali\Desktop\inventory>
```



To verify all works, start the development server by running:

```
python manage.py runserver
```

Then open http://127.0.0.1:8000

Note: runserver is a development server. It's convenient as it restarts automatically whenever you save a file, but it isn't built to handle real traffic and should never be used in production.

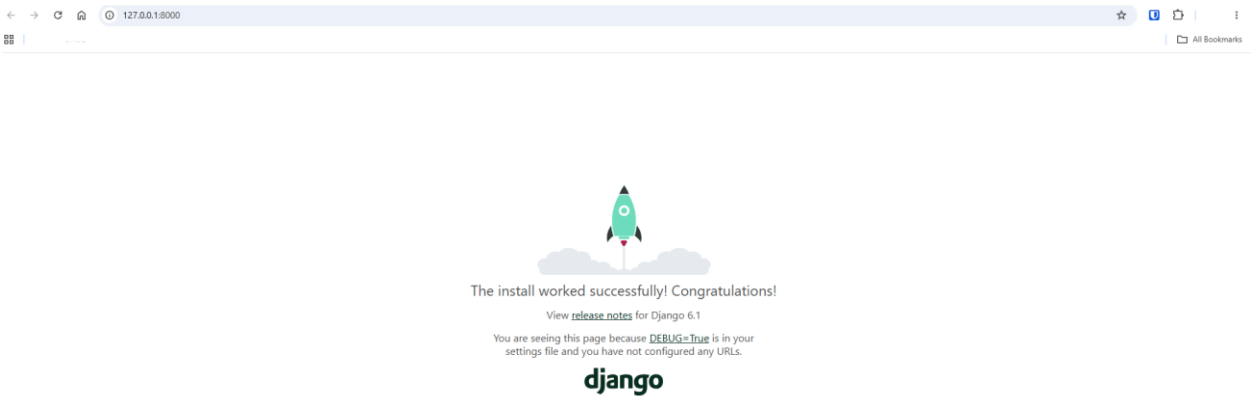
You can stop the server for now by pressing: CTRL + C

```
(venv) C:\Users\Ali\Desktop\Inventory_Agent>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
 auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
September 05, 2026 - 19:34:42
Django version 6.1.1, using settings 'config.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server inste
ad.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/howto/deployment/
[05/Sep/2026 19:34:43] "GET / HTTP/1.1" 200 12149
```



Step 4: The Database and Migrations

Our agent needs somewhere to keep inventory. Django comes with SQLite already configured, so we don't install or configure a database server.

Before we add our own inventory table, we need to create Django's built-in ones. Django ships with features that need database storage: user accounts, permissions, and sessions.

What is a migration?

A migration is a Python file describing a change to your database structure: "create a table with these columns", "add a column to that table". Django generates them for you, then runs them.

Why not just write SQL? Because migrations are versioned and repeatable. Everyone running migrate gets an identical database. When we later add a field to our inventory, Django writes a new migration, and everyone applies it in the same order. It's version control for database structure.

Django's built-in apps already ship their own migrations. We just need to run them.

In CMD run: python manage.py migrate

```
(venv) C:\Users\Ali\Desktop\Inventory_Agent>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

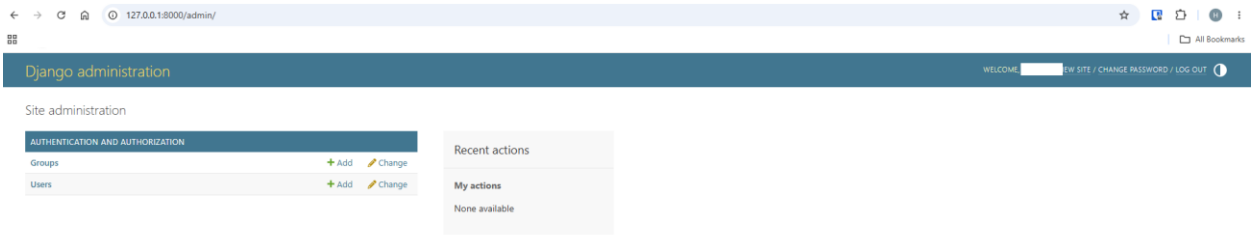
The migrate command just created Django's user table, so now we can make a user. This account logs into Django's admin panel, which we'll use to look at our inventory

In CMD run: python manage.py createsuperuser then create the needed credentials

```
(venv) C:\Users\Ali\Desktop\Inventory_Agent>python manage.py createsuperuser
Username (leave blank to use '[redacted]'):  [redacted]
Email address: [redacted]
Password:
Password (again):
Superuser created successfully.
```

To check it worked run: python manage.py runserver and open http://127.0.0.1:8000/admin

You should be able to log in with the account you just made.



Notes:

- db.sqlite3 is your entire database in one file. Delete it and you lose everything; you'd re-run migrate and createsuperuser to start fresh. That makes it a useful reset button when experimenting.
- Never commit db.sqlite3 to git. It's local data and it will cause conflicts.

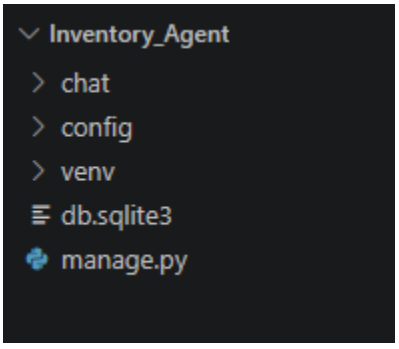
Step 5: Create the App and the Item Model

An "app" in Django is a folder holding one feature of your site. A project can have many apps: a blog app, a shop app, a user's app. Ours is small enough to need only one, which we'll call chat. It'll hold our inventory table, our tool functions, our agent loop, and our chat page. The split between "project" (config) and "app" (chat) is Django's way of separating configuration from features.

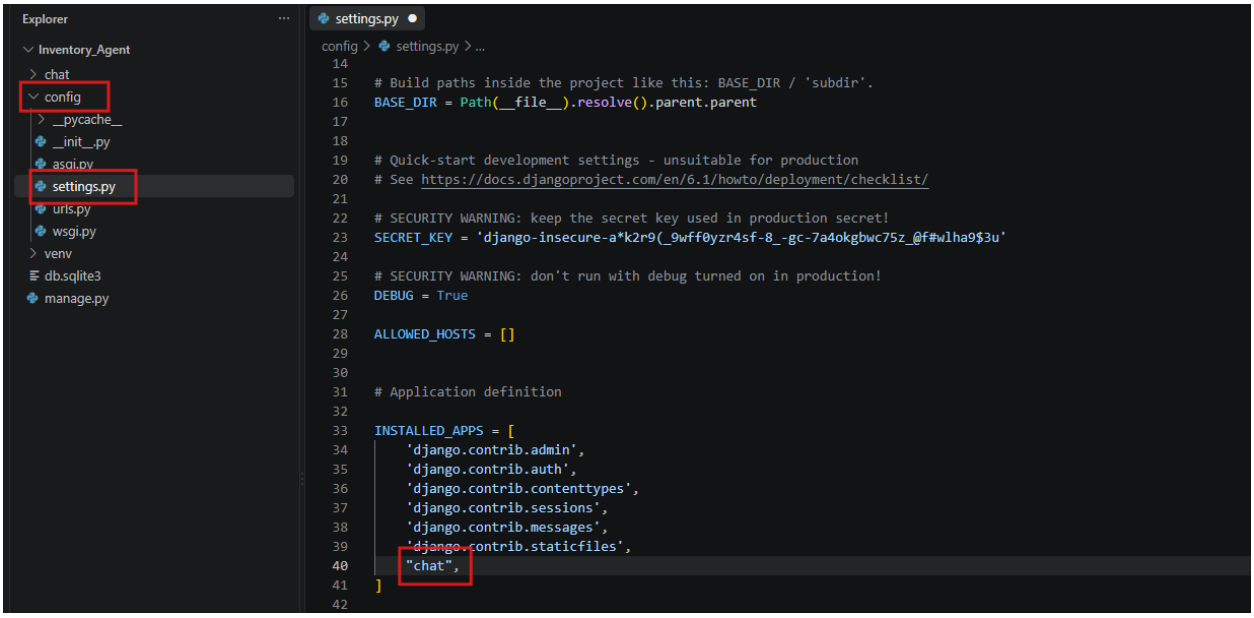
A model is a Python class that describes a database table. You write the class; Django writes the SQL. One class equals one table, one class attribute equals one column. This is the core idea of Django's database layer.

To create the app, run in CMD: python manage.py startapp chat

```
(venv) C:\Users\Ali\Desktop\Inventory_Agent>python manage.py startapp chat
```



Creating the folder isn't enough, Django only looks at apps listed in its settings. Open config/settings.py, find the INSTALLED_APPS list (around line 33), and check "chat" at the end.



Check chat/models.py code.

Now to show the model in the admin panel: check chat/admin.py.

Step 6: Create the Table and Add Your First Items

In Step 5 we described a table in Python. Now we turn that description into a real table, in two commands:

- "Makemigrations" reads your models, compares them to what it made last time, and writes a Python file describing the difference. It does not touch the database.
- **"**migrate**"** takes those files and executes them against the database.

To generate the migration, in CMD run: python manage.py makemigrations

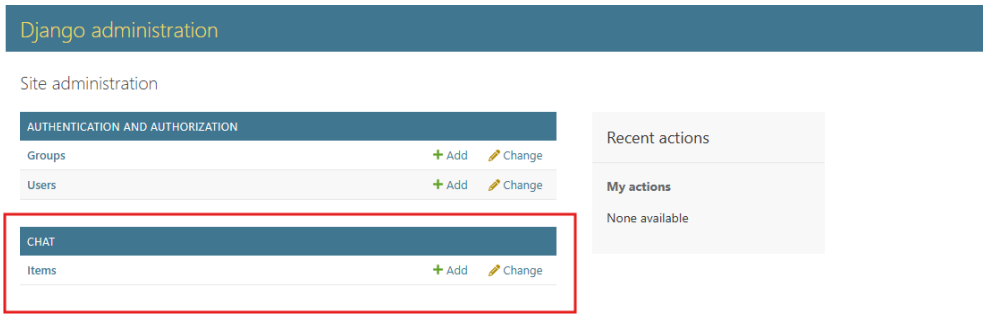
```
(venv) C:\Users\Ali\Desktop\Inventory_Agent>python manage.py makemigrations
Migrations for 'chat':
  chat\migrations\0001_initial.py
    + Create model Item
```

Then apply it by running: python manage.py migrate

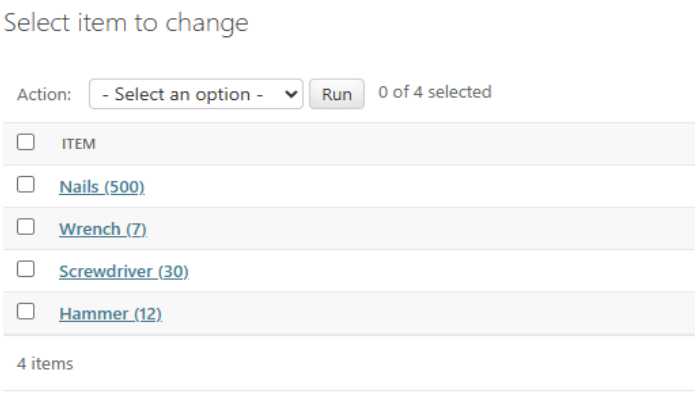
```
(venv) C:\Users\Ali\Desktop\Inventory_Agent>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, chat, contenttypes, sessions
Running migrations:
  Applying chat.0001_initial... OK
```

We will now add some items by hand, run: python manage.py runserver and go to http://127.0.0.1:8000/admin

There's now a CHAT section with Items in it. Click Add, and enter a few:



For the sake of this tutorial, those are the items we decided to add. Feel free to choose your own.



Notes:

- You now have a complete, working inventory application with zero AI in it. We have done this before!
- Any time you change models.py from now on, you run makemigrations then migrate.
- The admin panel is a permanent debugging tool for this project. When the agent claims it added something, this is where you check whether it did.

Step 7: Write the Tool Functions

Check tools.py.

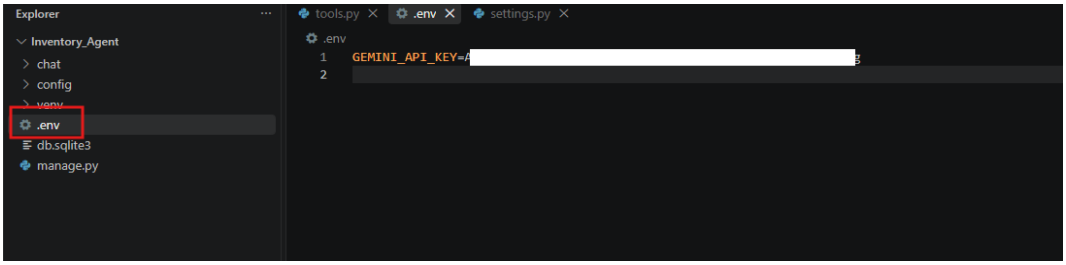
Step 8: Store Your API Key and Make Your First Gemini Call

Time to bring in the API key from Step 1. Two rules first:

1. Never put a key in your code. Code gets committed, shared, screenshotted and pasted into chats. Keys in code end up on GitHub, where bots scrape them within minutes.
2. Keys go in a .env file, which is a plain text file of NAME=value lines that your program reads at startup. It stays on your machine and never enters version control. This is why we installed python-dotenv back in Step 2.

Inside your main project folder, create the file ".env".

Inside it paste: GEMINI_API_KEY=<your-actual-key-here> (the one you saved earlier, drop < >)



To load the key when django starts, go to config/settings.py, near the top you'll find these two lines:

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
```

Replace them with:

```
from pathlib import Path
from dotenv import load_dotenv
BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")
```

Step 9: Describe Your Tools to the Model

The model can't see your Python code. It has no idea check_stock exists. So, we send it a description of each function alongside the user's message: the name, what it does, and what arguments it takes, written as JSON.

Check chat/tools.py after adding “declaration.json” below the functions.

Step 10: The Agent Loop

Everything so far was preparation. This is the agent.

Here's the whole idea before any code:

1. Send the user's message to the model, along with the tool descriptions.
2. The model replies. Its reply is either text (it's done) or a request to call a function (it needs something).
3. When the model finally replies with only text, that text is your answer.

That loop is the entire difference between a chatbot and an agent. A chatbot does one round trip. An agent keeps going until the model stops asking for things.

Notice who does what. The model never touches your database. It produces a message meaning "please run check_stock with name='Hammer'". The model is a decision-maker with no hands; your code is the hands.

Check inside the chat folder, “agent.py”.

Note:

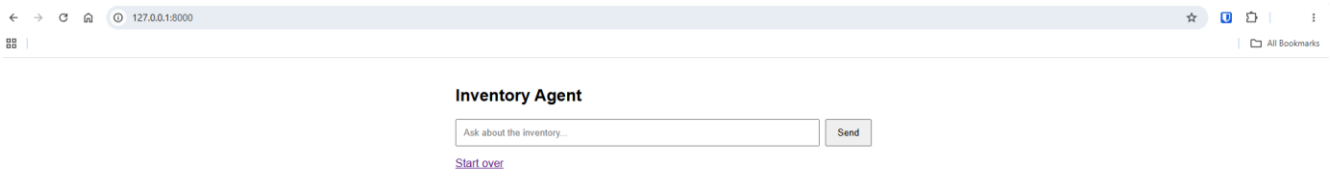
One thing to stop at in the “agent.py” code is the SYSTEM_PROMPT.

Remember that you are working with a general purpose LLM. The system prompt allows you to introduce it to the particular role it plays in your agent, what rules it must follow and any other context you seem is relevant for its specific job. Writing good prompts is an entire field, prompt engineers iterate over hundreds of different variants to find the one that outputs the best results. If you want to read more about prompt engineering check this link.

Another interesting note when it comes to designing good Agents is “Guardrails” which are safety measures you take to ensure your LLM doesn’t answer malicious questions or take actions it must not take. You will notice the final line in the system prompt under the rules section: “Do not answer any question unrelated to the inventory task.” This is a guardrail to ensure your model doesn’t answer questions unrelated to the inventory. Try asking the agent “Who is Messi” once with this guardrail and once after deleting it.

Step 11: The Chat Page

1. Check chat/views.py .
2. Check the folder “templates” under the chat folder and the file chat.html inside it.
3. Check config/urls.py
4. In CMD run: python manage.py runserver and go to http://127.0.0.1:8000/, you should see a simple chat interface.



Test your agent by asking it questions, you can pen the admin panel in another tab and confirm the numbers match (you may need to refresh the admin page):

1. What do we have in stock?
2. Add 15 hammers
3. How many do we have now?
4. Remove 3 of them

Open the admin panel in another tab and confirm the numbers match

End of Tutorial.